Sniper Bot: Paper-Trading Setup Guide

iPad to write code, Android (Termux) to run it, Cloudflare + Helius for detection, a Telegram Mini App to control it. Cost: $0 for this phase.

Scope. This phase is **paper trading only**. The code reads signals, runs a safety gate, simulates entries with pessimistic slippage, applies stop-loss and time-stop rules, and logs results. It holds **no private keys and sends no transactions**. Build a live executor only if the paper results justify it. This is engineering guidance, not financial advice. Most snipers lose money.

Items marked **[verify]** (free-tier limits, menu names, endpoints) change over time. Check the provider's current docs if something does not match.

1. How it fits together

```
Solana
  |
Helius webhook  (filters by wallet / program you choose)
  | HTTPS POST + Authorization header
Cloudflare Worker  (free)  ->  KV queue  ->  Telegram alerts
  ^                                  |
  | heartbeat / kill switch          | phone polls /signals every 3 s
  |                                  v
Android phone (Termux) ----> gate (RPC checks) -> paper position
                                     |              |
                                     |          exit rules (stop, time-stop,
                                     |          trail, take-profit, liquidity)
                                     v
                            logs/paper.jsonl -> node src/report.js
Telegram Mini App (served by the Worker at /app): status, halt, results, rules
iPad: edit code -> push to private GitHub repo -> Android: git pull
```

Known limitation. Cloudflare KV is eventually consistent, so a signal can reach the phone up to about a minute late. The bot logs this delay for every signal (`lagMs`), so you can see its effect on your results. It is acceptable for testing the gate and exit rules. It is not acceptable for a live executor, which should receive webhooks directly (see section 10).

2. Create the accounts (iPad, Safari, about 15 minutes)

1. **GitHub:** create a **private** repository named `sniper-bot`.
2. **Cloudflare:** create a free account at dash.cloudflare.com.
3. **Telegram:** open @BotFather, send `/newbot`, and copy the bot token. Send any message to your new bot, then open `https://api.telegram.org/bot<TOKEN>/getUpdates` and copy the numeric `chat.id` (in a private chat this is also your Telegram user id, which the Mini App uses to let only you in).
4. **Helius:** create a free account and copy your API key. Your RPC URL is `https://mainnet.helius-rpc.com/?api-key=YOUR_KEY`.
5. **Make two random secrets** (32+ characters each) with a password manager: **APP_KEY** (phone to Worker) and **HELIUS_AUTH** (Helius to Worker). Keep them in a note you can paste from.

3. Deploy the Worker (Cloudflare dashboard, no command line)

1. Workers & Pages > Create > Hello World worker. Name it `sniper-relay`. Deploy.
2. Open the worker > **Edit code**. Delete the sample and paste the full contents of `worker/index.js`, then Deploy. Easiest on the iPad: open the file in your GitHub repo in Safari, tap **Raw**, select all, copy. (Do not copy from the PDF appendix; use it for reference only.)
3. Storage & databases > KV > Create a namespace named `sniper-kv`.
4. Worker > Settings > Bindings > Add > KV namespace. Variable name **KV** (exactly), choose `sniper-kv`.

5. Worker > Settings > Variables and Secrets > add four **secrets**: `APP_KEY`, `HELIUS_AUTH`, `TG_TOKEN`, `TG_CHAT`.
6. Worker > Settings > Triggers > Cron Triggers > add `* * * * *` (runs the heartbeat watchdog).
7. Copy the worker URL (`https://sniper-relay.YOURNAME.workers.dev`). **[verify menu names]**

Free-tier math [verify]. KV free limits are small (about 1,000 writes/day, 100,000 reads/day). The phone polls every 3 s (two reads per poll, about 58,000 reads/day), writes a heartbeat every 3 min (about 480 writes/day), and writes again only when a new signal arrives or you change a setting. Keep your Helius filter narrow so signals stay well under the write limit.

4. Create the Helius webhook

1. Helius dashboard > Webhooks > New webhook. Network: mainnet. Type: **Enhanced**.
2. Webhook URL: `https://sniper-relay.YOURNAME.workers.dev/hook`
3. Authentication header: paste your **HELIUS_AUTH** value.
4. Account addresses: **start narrow**. Use a few wallets you want to track, or one program (for example a DEX program you want migrations from). Raw launch streams are huge and will burn free credits. Watch credit use for a day.
5. Save, then check delivery in the Helius webhook logs and the Worker logs.

Tune this once. `extractMints()` in the Worker turns a webhook payload into token mints. The default takes any non-SOL, non-stablecoin mint in `tokenTransfers`. Look at a few real payloads in the Worker logs and adapt it to the event you care about.

5. Prepare the Android phone (Termux)

1. Install **Termux**, **Termux:API** and **Termux:Boot** from F-Droid or the official GitHub releases (not the Play Store build).
2. In Termux: `pkg update && pkg upgrade` then `pkg install nodejs-lts git tmux termux-api unzip`.
3. Settings > Apps > Termux (and Termux:Boot, Telegram) > Battery > **Unrestricted**. Turn Adaptive Battery and Battery Saver off.
4. Developer options > **Disable child process restrictions** (Android 14+). Without it Android's phantom-process killer can end your bot silently.
5. Settings > Network > Private DNS > `one.one.one.one`.
6. Stay in the same user profile. On GrapheneOS, apps in other profiles stop when you switch away.
7. Keep the phone on a charger. Leave the Termux notification in place (it is the foreground service).

6. Get the code onto the phone and GitHub

You build on the iPad, but the first copy has to come from this chat. Open this conversation in the Claude app **on the Android phone** and download `sniper-bot.zip` to Downloads.

Create a GitHub token (Safari): Settings > Developer settings > Fine-grained tokens > only the `sniper-bot` repo, **Contents: Read and write**, 90-day expiry. Then in Termux:

```
termux-setup-storage
cd ~ && unzip ~/storage/downloads/sniper-bot.zip && cd sniper-bot
cp .env.example .env && nano .env        # fill RPC_URL, WORKER_URL, APP_KEY
git config --global credential.helper store
git init -b main && git add . && git commit -m "initial"
git remote add origin https://github.com/YOUR_NAME/sniper-bot.git
git push -u origin main                  # username = GitHub name, password = the token
```

.env stays on the phone. It is in `.gitignore`. Never type keys into files you edit on the iPad or commit to GitHub.

7. Daily workflow: write on iPad, run on Android

1. **iPad:** open github.com/YOU/sniper-bot in Safari and press `.` to get the browser VS Code editor (free), or clone the repo in **Working Copy** and edit with Textastic.
2. Make a small change, commit, push.
3. **Android (Termux):** `cd ~/sniper-bot && ./run.sh`. It pulls, then (re)starts the bot inside tmux with a wake lock.
4. Watch it: `tmux attach -t bot` (detach with Ctrl-b then d) or `tail -f logs/paper.jsonl`.
5. Optional: run `pkg install openssh` in Termux and `sshd` (port 8022), then use Blink Shell or Termius on the iPad to restart the bot and read logs over the same Wi-Fi.

Auto-start after reboot (Termux:Boot). Open Termux:Boot once, then:

```
mkdir -p ~/.termux/boot
cat > ~/.termux/boot/start.sh <<'EOF'
#!/data/data/com.termux/files/usr/bin/bash
termux-wake-lock
cd ~/sniper-bot && ./run.sh
EOF
chmod +x ~/.termux/boot/start.sh
```

8. Verify the pipeline end to end

1. Start the bot. You should see a `start` line and a Telegram message "Bot started (paper mode)".
2. Send a fake webhook (from Termux). BONK is used only as an example mint; it will probably fail the gate, which is fine, since the point is seeing the full path work:

```
curl -X POST "$WORKER_URL/hook" \
  -H "authorization: $HELIUS_AUTH" -H "content-type: application/json" \
  -d '[{"signature":"test1","tokenTransfers":[{"mint":"DezXAZ8z7PnrnRJjz3wXBoRgixCa6xjnB7YaB1pPB263"}]}]'
```

Within about 3 seconds the log should show a `gate` line with pass/fail reasons and `lagMs`. (Set WORKER_URL and HELIUS_AUTH in your shell for this command.)

3. **Kill switch test:** `touch HALT` blocks new entries. Remote: `curl -X POST "$WORKER_URL/halt?on=1" -H "x-key: $APP_KEY"` (use on=0 to resume).
4. **Watchdog test:** stop the bot, wait about 12 minutes, and confirm a "Phone silent" Telegram alert arrives.
5. **Report:** `node src/report.js` prints signals gated, relay lag, win rate, mean/median PnL and results by exit reason.

9. Telegram Mini App control panel

A Mini App is a web page that opens inside Telegram. Here it is your **remote control and dashboard**: phone online/offline, a halt button, open paper positions, results, recent gate checks, and editable exit rules.

What it is not. Telegram suspends a Mini App when you close it, so it cannot run the scanner or trader. The engine keeps running on the Android phone in Termux. The panel shows the phone's last heartbeat (every 3 minutes) and sends commands that the phone picks up within a few seconds.

1. **Update the Worker:** open the worker in the Cloudflare dashboard > Edit code, replace everything with the new `worker/index.js` (copy it from GitHub > Raw) and Deploy. Check that the `TG_CHAT` secret is **your own numeric Telegram user id**. The panel only opens for that id.
2. **Update the phone:** push the new project files to GitHub, then in Termux run `cd ~/sniper-bot && ./run.sh`.
3. **Add the panel to your bot:** in Telegram open @BotFather > /mybots > your bot > Bot Settings > Menu Button > Configure menu button. Send the URL `https://sniper-relay.YOURNAME.workers.dev/app` and a

button title such as Panel. **[verify BotFather menu names]**

4. Open your bot chat and tap **Panel**. Do not open the /app link in a normal browser: it only works inside Telegram.

What you can do in the panel

- **Halt / Resume new entries:** the phone checks the flag on every poll. Existing paper positions still follow their exit rules.
- **Edit exit rules:** stop-loss, time-stop, band, max hold, take-profit, trailing stop, liquidity drop. The Worker range-checks every value and the phone logs each change as `rule_change`.
- **Read results:** win rate, mean and median PnL, results by exit reason, relay lag.

How it is secured

- Every `/api` call must carry Telegram's signed `initData`. The Worker verifies the signature with your bot token, rejects data older than 24 hours, and accepts only your user id.
- The page itself contains no secrets. Phone endpoints still require APP_KEY. The Mini App never sees APP_KEY or any key.
- No signing, wallet or trading functions exist in the panel. In the paper phase it can only halt and tune rules.

Troubleshooting

- **"Error: HTTP 401"**: TG_CHAT does not equal your user id, TG_TOKEN is wrong, or the page was opened outside Telegram.
- **Phone shows "never seen"**: the bot is not running, or APP_KEY / WORKER_URL in `.env` do not match.
- **Phone shows "offline"**: no heartbeat for 8 minutes. Check battery settings and `tmux attach -t bot`.
- **Rule edit not applied**: wait a few seconds, then look for a `rule_change` line in the log.

10. Paper-phase rules and the go / no-go decision

Setting (src/config.js)	Default	Meaning
stopLoss	25%	Exit when executable value is 25% below entry
timeStopSec / band	180 s, -10% to +15%	Exit if sideways in the band this long with no new high
maxHoldMin	30	Exit the remainder after 30 minutes
takeProfit	+100%	Sell half (recoup principal)
trailArm / trail	+50% / 20%	Trailing stop once armed
liqDrop	30%	Exit if liquidity drops 30% from entry
ENTRY_SLIP / EXIT_SLIP	5% / 8%	Pessimistic fills (set in .env). Rule rows above can also be edited in the Mini App.
MAX_TOP10_PCT	35%	Reject if top-10 wallet holders exceed this share

Run for **at least two weeks and 200 closed paper trades** before spending anything. Proceed only if all hold:

- Mean and **median** PnL are positive after the built-in slippage penalties (a few big winners can hide a negative median).
- The result survives doubling ENTRY_SLIP and EXIT_SLIP.
- `no_quote_total_loss` exits (likely rugs or vanished pools) are a small share of trades.
- Relay lag is understood. If median lag is large, the live version must receive webhooks directly.

If the numbers are negative, stop. That is a valid and cheap result. If they are positive, the next step is a small VPS (Frankfurt or Singapore, about $5/month) running a live executor that receives the Helius webhook directly, signs with a capped burner key, and sends through Helius Sender and Jito. That executor is not part of this package.

11. Security checklist

- No private keys exist in this project. Keep it that way until the live phase.
- Repo is private. `.env`, logs and key files are git-ignored.
- GitHub token is fine-grained, limited to this repo, and expires.
- Worker secrets (APP_KEY, HELIUS_AUTH, TG_TOKEN) live only in Cloudflare and `.env`.
- The Worker rejects any request without the correct header or valid Telegram initData. Rotate APP_KEY if you ever paste it somewhere public.
- Zero dependencies means no npm supply-chain risk in this phase. Think hard before adding packages.

Appendix: source files

Files are in `sniper-bot.zip`. Listings below are for reference and for pasting edits.

`package.json`

```
{
  "name": "sniper-bot",
  "private": true,
  "type": "module",
  "engines": { "node": ">=18" },
  "scripts": {
    "start": "node src/main.js",
    "report": "node src/report.js"
  }
}
```

`.gitignore`

```
.env
logs/
node_modules/
HALT
*.key
keystore*
```

`.env.example`

```
# Copy to .env on the ANDROID device only. Never commit .env.
RPC_URL=https://mainnet.helius-rpc.com/?api-key=YOUR_HELIUS_KEY
WORKER_URL=https://sniper-relay.YOURNAME.workers.dev
APP_KEY=long-random-string-same-as-worker-secret
# Optional tuning
# POLL_MS=3000
# PRICE_MS=5000
# MAX_OPEN=5
# MIN_LIQ_USD=15000
# MAX_TOP10_PCT=35
# ENTRY_SLIP=0.05
# EXIT_SLIP=0.08
```

`run.sh`

```
#!/data/data/com.termux/files/usr/bin/bash
# Pull the latest code and (re)start the bot in tmux. Run on the Android device.
set -e
cd "$(dirname "$0")"
termux-wake-lock || true
git pull --ff-only
tmux kill-session -t bot 2>/dev/null || true
tmux new-session -d -s bot 'while true; do node src/main.js; sleep 2; done'
echo "Started. Attach: tmux attach -t bot   Logs: tail -f logs/paper.jsonl"
```

`src/config.js`

```
import { readFileSync, existsSync } from "node:fs";

// Minimal .env loader (no dependencies).
if (existsSync(".env")) {
  for (const line of readFileSync(".env", "utf8").split("\n")) {
    const m = line.match(/^\s*([A-Z0-9_]+)\s*=\s*(.*?)\s*$/);
    if (m && !(m[1] in process.env)) process.env[m[1]] = m[2].replace(/^["']|["']$/g, "");
  }
}

const need = (k) => {
  const v = process.env[k];
  if (!v) throw new Error(`Missing env var ${k} (see .env.example)`);
  return v;
};
const num = (k, d) => (process.env[k] ? Number(process.env[k]) : d);

export const cfg = {
  rpcUrl: need("RPC_URL"),
  workerUrl: need("WORKER_URL").replace(/\/$/, ""),
  appKey: need("APP_KEY"),
  pollMs: num("POLL_MS", 3000),
  priceMs: num("PRICE_MS", 5000),
  heartbeatMs: 3 * 60 * 1000,
  maxOpen: num("MAX_OPEN", 5),
  minLiqUsd: num("MIN_LIQ_USD", 15000),
```

```
  // Pessimistic paper fills: you will not get the screen price.
  entrySlip: num("ENTRY_SLIP", 0.05),
  exitSlip: num("EXIT_SLIP", 0.08),
  gate: { maxTop10Pct: num("MAX_TOP10_PCT", 35) },
  // Exit rules (percent unless noted). Tune these from the iPad.
  rules: {
    stopLoss: 25,      // exit at -25% from entry
    bandDown: 10,      // time-stop band: -10% ...
    bandUp: 15,        // ... to +15% of entry
    timeStopSec: 180,  // exit if sideways in band this long with no new high
    maxHoldMin: 30,    // exit remainder after 30 min
    takeProfit: 100,   // sell half at +100%
    trailArm: 50,      // arm trailing stop at +50%
    trail: 20,         // exit on 20% drawdown from peak once armed
    liqDrop: 30,       // exit if liquidity falls 30% from entry
    noQuoteTicks: 12,  // consecutive missing quotes before counting a total loss
  },
};
```

src/log.js

```
import { appendFileSync, mkdirSync } from "node:fs";

mkdirSync("logs", { recursive: true });

// One JSON object per line; read later with src/report.js.
export function log(ev, data = {}) {
  const row = { ts: new Date().toISOString(), ev, ...data };
  console.log(JSON.stringify(row));
  try {
    appendFileSync("logs/paper.jsonl", JSON.stringify(row) + "\n");
  } catch {
    /* never crash on logging */
  }
}
```

src/rpc.js

```
import { cfg } from "./config.js";

let id = 0;

export async function rpc(method, params) {
  const r = await fetch(cfg.rpcUrl, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({ jsonrpc: "2.0", id: ++id, method, params }),
    signal: AbortSignal.timeout(8000),
  });
  const j = await r.json();
  if (j.error) throw new Error(`${method}: ${j.error.message}`);
  return j.result;
}
```

src/api.js

```
import { cfg } from "./config.js";

const headers = { "x-key": cfg.appKey, "content-type": "application/json" };

export async function getSignals(since) {
  const r = await fetch(`${cfg.workerUrl}/signals?since=${since}`, {
    headers,
    signal: AbortSignal.timeout(8000),
  });
  if (!r.ok) throw new Error(`signals HTTP ${r.status}`);
  return r.json();
}

export const notify = (text) =>
  fetch(`${cfg.workerUrl}/notify`, {
    method: "POST",
    headers,
    body: JSON.stringify({ text }),
    signal: AbortSignal.timeout(8000),
  }).catch(() => {});

export const heartbeat = (snapshot) =>
  fetch(`${cfg.workerUrl}/heartbeat`, {
    method: "POST",
    headers,
    body: JSON.stringify(snapshot),
    signal: AbortSignal.timeout(8000),
  }).catch(() => {});
```

src/price.js

```
// Free price + liquidity source for the paper phase (DexScreener).
// New pairs can take a short while to be indexed: a null result means "not yet".
export async function getQuote(mint) {
  try {
    const r = await fetch(`https://api.dexscreener.com/latest/dex/tokens/${mint}`, {
      signal: AbortSignal.timeout(8000),
    });
    if (!r.ok) return null;
    const j = await r.json();
    const pairs = (j.pairs ?? []).filter((p) => p.chainId === "solana" && Number(p.priceUsd) > 0);
    if (!pairs.length) return null;
    pairs.sort((a, b) => (b.liquidity?.usd ?? 0) - (a.liquidity?.usd ?? 0));
    const p = pairs[0];
    return { price: Number(p.priceUsd), liqUsd: p.liquidity?.usd ?? 0, pair: p.pairAddress };
  } catch {
    return null;
  }
}
```

src/stats.js

```
import { readFileSync, existsSync } from "node:fs";

export function readRows(file = "logs/paper.jsonl") {
  if (!existsSync(file)) return [];
  return readFileSync(file, "utf8").split("\n").filter(Boolean).map((l) => {
    try { return JSON.parse(l); } catch { return null; }
  }).filter(Boolean);
}

const sum = (a) => a.reduce((s, x) => s + x, 0);
const mean = (a) => (a.length ? sum(a) / a.length : 0);
const median = (a) => (a.length ? a[Math.floor(a.length / 2)] : 0);

// Pure summary of paper results; used by report.js and the Mini App heartbeat.
export function computeStats(rows) {
  const gates = rows.filter((r) => r.ev === "gate");
  const closed = rows.filter((r) => r.ev === "paper_close");
  const pnls = closed.map((r) => r.pnl).sort((a, b) => a - b);
  const lags = gates.map((g) => g.lagMs).filter(Number.isFinite).sort((a, b) => a - b);
  const by = {};
  for (const r of closed) (by[r.reason] ??= []).push(r.pnl);
  const byReason = {};
  for (const [k, v] of Object.entries(by)) byReason[k] = { n: v.length, mean: Number(mean(v).toFixed(1)) };
  return {
    gated: gates.length,
    passed: gates.filter((g) => g.pass).length,
    closed: closed.length,
    winRate: pnls.length ? (pnls.filter((x) => x > 0).length / pnls.length) * 100 : 0,
    mean: Number(mean(pnls).toFixed(2)),
    median: Number(median(pnls).toFixed(2)),
    best: pnls.length ? pnls.at(-1) : null,
    worst: pnls.length ? pnls[0] : null,
    lagMedian: lags.length ? median(lags) : null,
    lagP90: lags.length ? lags[Math.floor(lags.length * 0.9)] : null,
    byReason,
  };
}
```

src/gate.js

```
import { rpc } from "./rpc.js";

const SYSTEM_PROGRAM = "11111111111111111111111111111111";

// Returns { pass, reasons[], top10Pct }. Any failed check rejects the token.
export async function runGate(mint, gateCfg) {
  const reasons = [];

  // 1) Authorities must be revoked.
  const acc = await rpc("getAccountInfo", [mint, { encoding: "jsonParsed" }]);
  const info = acc?.value?.data?.parsed?.info;
  if (!info) return { pass: false, reasons: ["not a parsed SPL mint"], top10Pct: null };
  if (info.mintAuthority) reasons.push("mint authority not revoked");
  if (info.freezeAuthority) reasons.push("freeze authority not revoked");

  // 2) Holder concentration, counting only wallet-owned accounts.
  //    Pool vaults / bonding curves are owned by programs (PDAs), so they are excluded.
  const supply = Number((await rpc("getTokenSupply", [mint]))?.value?.uiAmount ?? 0);
  const largest = ((await rpc("getTokenLargestAccounts", [mint]))?.value ?? []).slice(0, 20);
  const jp = { encoding: "jsonParsed" };
  const tokenAccs = largest.length
```

```
    ? (await rpc("getMultipleAccounts", [largest.map((a) => a.address), jp]))?.value ?? []
    : [];
  const owners = tokenAccs.map((a) => a?.data?.parsed?.info?.owner ?? null);
  const ownerKeys = [...new Set(owners.filter(Boolean))];
  const noData = { encoding: "base64", dataSlice: { offset: 0, length: 0 } };
  const ownerInfo = ownerKeys.length
    ? (await rpc("getMultipleAccounts", [ownerKeys, noData]))?.value ?? []
    : [];
  const isWallet = new Map(ownerKeys.map((k, i) => [k, ownerInfo[i]?.owner === SYSTEM_PROGRAM]));

  const wallets = largest
    .map((a, i) => ({ amt: Number(a.uiAmount ?? 0), wallet: isWallet.get(owners[i]) === true }))
    .filter((h) => h.wallet)
    .slice(0, 10);
  const top10Pct = supply > 0 ? (wallets.reduce((s, h) => s + h.amt, 0) / supply) * 100 : 100;
  if (top10Pct > gateCfg.maxTop10Pct) {
    reasons.push(`top10 wallets hold ${top10Pct.toFixed(1)}% (> ${gateCfg.maxTop10Pct}%)`);
  }

  return { pass: reasons.length === 0, reasons, top10Pct: Number(top10Pct.toFixed(1)) };
}
```

src/paper.js

```
import { cfg } from "./config.js";
import { getQuote } from "./price.js";
import { log } from "./log.js";
import { notify } from "./api.js";

const R = cfg.rules;
const positions = new Map(); // mint -> position

export const openCount = () => positions.size;
export const isOpen = (mint) => positions.has(mint);

// Compact view of open positions for the Mini App.
export const snapshot = () =>
  [...positions.values()].map((p) => ({
    mint: p.mint,
    pct: p.lastPct === undefined ? null : Number(p.lastPct.toFixed(1)),
    ageSec: Math.round((Date.now() - p.openedAt) / 1000),
    half: p.tookHalf,
  }));

export async function openPaper(mint, sig) {
  if (positions.size >= cfg.maxOpen) return log("skip_maxopen", { mint });
  const q = await getQuote(mint);
  if (!q) return log("skip_noquote", { mint });
  if (q.liqUsd < cfg.minLiqUsd) return log("skip_liquidity", { mint, liqUsd: q.liqUsd });

  const entry = q.price * (1 + cfg.entrySlip);
  positions.set(mint, {
    mint, sig, entry,
    peak: entry,
    openedAt: Date.now(),
    liq0: q.liqUsd,
    sidewaysSince: null,
    trailArmed: false,
    tookHalf: false,
    realized: 0,       // pct points already banked (weighted)
    misses: 0,
  });
  log("paper_open", { mint, entry, liqUsd: q.liqUsd });
  notify(`PAPER BUY ${mint}\nentry ${entry}\nliq $${Math.round(q.liqUsd)}`);
}

// Pure rule engine. Returns null, { partial }, or { exit } with reason and pct.
export function evaluate(p, q, now = Date.now()) {
  const px = q.price * (1 - cfg.exitSlip);          // pessimistic executable price
  const pct = (px / p.entry - 1) * 100;
  p.lastPct = pct;
  const newHigh = px > p.peak;
  if (newHigh) p.peak = px;

  if (pct <= -R.stopLoss) return { exit: "stop_loss", pct };
  if (q.liqUsd < p.liq0 * (1 - R.liqDrop / 100)) return { exit: "liquidity_drop", pct };
  if (now - p.openedAt >= R.maxHoldMin * 60000) return { exit: "max_hold", pct };

  if (pct >= R.trailArm) p.trailArmed = true;
  if (p.trailArmed && px <= p.peak * (1 - R.trail / 100)) return { exit: "trailing_stop", pct };

  if (!p.tookHalf && pct >= R.takeProfit) return { partial: "take_profit_half", pct };

  const inBand = pct >= -R.bandDown && pct <= R.bandUp;
  if (inBand && !newHigh) {
```

```
    p.sidewaysSince ??= now;
    if (now - p.sidewaysSince >= R.timeStopSec * 1000) return { exit: "time_stop", pct };
  } else {
    p.sidewaysSince = null;
  }
  return null;
}

function close(p, reason, pct) {
  const weight = p.tookHalf ? 0.5 : 1;
  const pnl = p.realized + weight * pct;
  positions.delete(p.mint);
  const heldSec = Math.round((Date.now() - p.openedAt) / 1000);
  log("paper_close", { mint: p.mint, reason, pnl: Number(pnl.toFixed(2)), heldSec });
  notify(`PAPER EXIT ${p.mint}\n${reason}  pnl ${pnl.toFixed(1)}%  held ${heldSec}s`);
}

let busy = false;
export async function tick() {
  if (busy) return;
  busy = true;
  try {
    for (const p of [...positions.values()]) {
      const q = await getQuote(p.mint);
      if (!q) {
        // No price for ~1 minute is treated as a rug: conservative accounting.
        if (++p.misses >= R.noQuoteTicks) close(p, "no_quote_total_loss", -100);
        continue;
      }
      p.misses = 0;
      const res = evaluate(p, q);
      if (!res) continue;
      if (res.partial) {
        p.tookHalf = true;
        p.realized += 0.5 * res.pct;
        log("paper_partial", { mint: p.mint, reason: res.partial, pct: Number(res.pct.toFixed(1)) });
      } else {
        close(p, res.exit, res.pct);
      }
    }
  } finally {
    busy = false;
  }
}
```

src/main.js

```
import { existsSync } from "node:fs";
import { cfg } from "./config.js";
import { getSignals, heartbeat, notify } from "./api.js";
import { runGate } from "./gate.js";
import { openPaper, tick, snapshot, isOpen } from "./paper.js";
import { readRows, computeStats } from "./stats.js";
import { log } from "./log.js";

let since = -1;              // -1 = first poll only learns the current sequence
const seen = new Set();      // mints already handled this run
const recentGates = [];      // last 10 gate results, shown in the Mini App

// Apply rule overrides set from the Mini App (server already range-checks them).
function applyRules(rules) {
  for (const k of Object.keys(cfg.rules)) {
    const v = rules?.[k];
    if (Number.isFinite(v) && v !== cfg.rules[k]) {
      log("rule_change", { key: k, from: cfg.rules[k], to: v });
      cfg.rules[k] = v;
    }
  }
}

async function handle(s) {
  const lagMs = Date.now() - s.ts;   // relay delay: measure it, it is part of your real latency
  const g = await runGate(s.mint, cfg.gate);
  log("gate", { mint: s.mint, pass: g.pass, reasons: g.reasons, top10Pct: g.top10Pct, lagMs });
  recentGates.push({ mint: s.mint, pass: g.pass, reasons: g.reasons, ts: Date.now() });
  if (recentGates.length > 10) recentGates.shift();
  if (g.pass && !isOpen(s.mint)) await openPaper(s.mint, s.sig);
}

async function pollOnce() {
  const { signals = [], seq = since, halt = false, rules } = await getSignals(since);
  since = seq;
  applyRules(rules);
  const halted = halt || existsSync("HALT");   // kill switch: remote flag or local file
  for (const s of signals) {
```

```
    if (seen.has(s.mint)) continue;
    seen.add(s.mint);
    if (halted) { log("halted_skip", { mint: s.mint }); continue; }
    handle(s).catch((e) => log("error", { where: "handle", msg: String(e) }));
  }
}

async function pollLoop() {
  try {
    await pollOnce();
  } catch (e) {
    log("error", { where: "poll", msg: String(e) });
  }
  setTimeout(pollLoop, cfg.pollMs);
}

// Snapshot for the Mini App + watchdog. Written every 3 min (KV free tier has a daily write cap).
function sendHeartbeat() {
  try {
    heartbeat({ open: snapshot(), stats: computeStats(readRows()), gates: recentGates, rules: cfg.rules });
  } catch (e) {
    log("error", { where: "heartbeat", msg: String(e) });
  }
}

log("start", { pollMs: cfg.pollMs, paperOnly: true });
notify("Bot started (paper mode)");
pollLoop();
setInterval(() => tick().catch((e) => log("error", { where: "tick", msg: String(e) })), cfg.priceMs);
setInterval(sendHeartbeat, cfg.heartbeatMs);
sendHeartbeat();
process.on("SIGINT", () => { log("stop", {}); process.exit(0); });
```

src/report.js

```
// Usage: node src/report.js   -> summary of paper results from logs/paper.jsonl
import { readRows, computeStats } from "./stats.js";

const rows = readRows();
if (!rows.length) { console.log("No log yet."); process.exit(0); }
const s = computeStats(rows);

console.log(`Signals gated: ${s.gated}  passed: ${s.passed}`);
if (s.lagMedian !== null) console.log(`Relay lag ms  median: ${s.lagMedian}  p90: ${s.lagP90}`);
console.log(`Closed trades: ${s.closed}`);
if (s.closed) {
  console.log(`Win rate: ${s.winRate.toFixed(1)}%`);
  console.log(`Mean pnl: ${s.mean}%  Median: ${s.median}%`);
  console.log(`Best: ${s.best}%  Worst: ${s.worst}%`);
  console.log("By exit reason:");
  for (const [k, v] of Object.entries(s.byReason)) {
    console.log(`  ${k.padEnd(22)} n=${v.n}  mean=${v.mean}%`);
  }
}
```

worker/index.js

```
// Cloudflare Worker relay + Telegram Mini App panel (paste into the dashboard editor).
// Helius webhook -> KV queue -> phone polls /signals. Mini App at /app. Telegram alerts, watchdog, kill switch.
// Bindings: KV (KV namespace). Secrets: APP_KEY, HELIUS_AUTH, TG_TOKEN, TG_CHAT (your own numeric Telegram user id).
// Cron trigger: * * * * *

const WSOL = "So11111111111111111111111111111111111111112";
const USDC = "EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v";
const USDT = "Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB";
const IGNORE = new Set([WSOL, USDC, USDT]);
const QMAX = 50;                    // keep the last 50 signals
const HB_LATE_MS = 12 * 60 * 1000;  // alert if the phone is silent this long

// Editable from the Mini App. [min, max] per rule; anything else is rejected.
const RULE_LIMITS = {
  stopLoss: [1, 95], bandDown: [0, 50], bandUp: [0, 100], timeStopSec: [10, 3600],
  maxHoldMin: [1, 720], takeProfit: [10, 1000], trailArm: [10, 1000], trail: [1, 90], liqDrop: [5, 95],
};

const json = (o, status = 200) =>
  new Response(JSON.stringify(o), { status, headers: { "content-type": "application/json" } });

async function tg(env, text) {
  if (!env.TG_TOKEN || !env.TG_CHAT) return;
  await fetch(`https://api.telegram.org/bot${env.TG_TOKEN}/sendMessage`, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({ chat_id: env.TG_CHAT, text: String(text).slice(0, 3500) }),
  });
}

// ADAPT THIS after you have seen real payloads (Worker logs). Default: any non-SOL/stable token in transfers.
function extractMints(tx) {
  const out = new Set();
  for (const t of tx.tokenTransfers ?? []) if (t.mint && !IGNORE.has(t.mint)) out.add(t.mint);
  return [...out];
}

const readQ = async (env) => (await env.KV.get("queue", "json")) ?? { seq: 0, items: [] };
const readCtl = async (env) => (await env.KV.get("ctl", "json")) ?? { halt: false, rules: {} };

async function setHalt(env, on) {
  const ctl = await readCtl(env);
  ctl.halt = on;
  await env.KV.put("ctl", JSON.stringify(ctl));
  await tg(env, on ? "HALT set: no new entries" : "HALT cleared");
  return ctl;
```

```
}

function cleanRules(input) {
  const out = {};
  for (const [k, [lo, hi]] of Object.entries(RULE_LIMITS)) {
    const v = Number(input?.[k]);
    if (input && k in input && Number.isFinite(v) && v >= lo && v <= hi) out[k] = v;
  }
  return out;
}

// ---- Telegram Mini App auth: validates initData per Telegram's HMAC scheme, owner only ----
async function hmac(keyBytes, data) {
  const key = await crypto.subtle.importKey("raw", keyBytes, { name: "HMAC", hash: "SHA-256" }, false, ["sign"]);
  return new Uint8Array(await crypto.subtle.sign("HMAC", key, new TextEncoder().encode(data)));
}

export async function verifyInit(env, initData) {
  if (!initData || !env.TG_TOKEN || !env.TG_CHAT) return false;
  const params = new URLSearchParams(initData);
  const hash = params.get("hash");
  if (!hash) return false;
  params.delete("hash");
  const dcs = [...params.entries()].map(([k, v]) => `${k}=${v}`).sort().join("\n");
  const secret = await hmac(new TextEncoder().encode("WebAppData"), env.TG_TOKEN);
  const sig = [...(await hmac(secret, dcs))].map((b) => b.toString(16).padStart(2, "0")).join("");
  if (sig !== hash) return false;
  if (Date.now() / 1000 - Number(params.get("auth_date")) > 86400) return false;
  try {
    return String(JSON.parse(params.get("user")).id) === String(env.TG_CHAT);
  } catch {
    return false;
  }
}

const APP_HTML = `<!doctype html><html><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1,maximum-scale=1,viewport-fit=cover">
<title>Sniper Panel</title>
<script src="https://telegram.org/js/telegram-web-app.js"></script>
<style>
:root{--bg:var(--tg-theme-bg-color,#fff);--fg:var(--tg-theme-text-color,#111);--hint:var(--tg-theme-hint-color,#777);
--card:var(--tg-theme-secondary-bg-color,#f2f3f5);--btn:var(--tg-theme-button-color,#2481cc);--btnfg:var(--tg-theme-button-text-color,#fff)}
*{box-sizing:border-box}
body{margin:0;padding:12px 12px 24px;font:15px/1.4 -apple-system,system-ui,sans-serif;background:var(--bg);color:var(--fg)}
.card{background:var(--card);border-radius:12px;padding:12px;margin-bottom:10px}
h2{font-size:12px;text-transform:uppercase;letter-spacing:.05em;color:var(--hint);margin:0 0 8px}
.row{display:flex;justify-content:space-between;gap:8px;padding:3px 0}
.mono{font-family:ui-monospace,Menlo,monospace;font-size:12px}
.good{color:#1a9f4b}.bad{color:#d93025}.hint{color:var(--hint)}
button{background:var(--btn);color:var(--btnfg);border:0;border-radius:10px;padding:11px 14px;font-size:15px;width:100%;margin-top:8px}
button.red{background:#d93025;color:#fff}
.grid{display:grid;grid-template-columns:1fr 1fr;gap:8px}
input{width:100%;padding:8px;border-radius:8px;border:1px solid var(--hint);background:var(--bg);color:var(--fg);font-size:15px}
label{font-size:12px;color:var(--hint);display:block;margin-bottom:2px}
.small{font-size:12px;word-break:break-word}
</style></head><body>
<div class="card" id="status"></div>
<div class="card"><h2>Open paper positions</h2><div id="open"></div></div>
<div class="card"><h2>Results (paper)</h2><div id="stats"></div></div>
<div class="card"><h2>Recent gate checks</h2><div id="gates"></div></div>
<div class="card"><h2>Exit rules</h2><div class="grid" id="rules"></div><button id="save">Save rules</button></div>
<div class="hint small" style="text-align:center" id="foot"></div>
<script>
var tg = window.Telegram && window.Telegram.WebApp;
if (tg) { tg.ready(); tg.expand(); }
var init = tg ? tg.initData : "";
var RULES = [["stopLoss","Stop-loss %"],["timeStopSec","Time-stop sec"],["bandDown","Band down %"],["bandUp","Band up %"],
  ["maxHoldMin","Max hold min"],["takeProfit","Take-profit %"],["trailArm","Trail arm %"],["trail","Trail %"],["liqDrop","Liq drop %"]];
var rulesShown = false;
function $(id){ return document.getElementById(id); }
function el(tag, text, cls){ var e = document.createElement(tag); if (text !== undefined) e.textContent = text; if (cls) e.className = cls; return e; }
function row(a, b, cls){ var r = el("div", undefined, "row"); r.appendChild(el("span", a)); r.appendChild(el("span", b, cls)); return r; }
function api(path, method, body){
  return fetch(path, { method: method || "GET", headers: { "x-init": init, "content-type": "application/json" },
    body: body ? JSON.stringify(body) : undefined }).then(function(r){ if (!r.ok) throw new Error("HTTP " + r.status); return r.json(); });
}
function ago(ts){ var s = Math.round((Date.now() - ts) / 1000); if (s < 90) return s + "s ago"; if (s < 5400) return Math.round(s / 60) + "m ago"; return Math.round(s / 3600) + "h ago"; }
function short(m){ return m.slice(0, 4) + "..." + m.slice(-4); }
function pc(x){ return (x === null || x === undefined) ? "n/a" : (x > 0 ? "+" : "") + Number(x).toFixed(1) + "%"; }
function clear(id){ var e = $(id); e.textContent = ""; return e; }

function render(s){
  var hb = s.hb || {};
  var online = hb.ts && (Date.now() - hb.ts) < 8 * 60000;
  var st = clear("status");
  st.appendChild(row("Phone", online ? "online" : (hb.ts ? "offline" : "never seen"), online ? "good" : "bad"));
  st.appendChild(row("Last heartbeat", hb.ts ? ago(hb.ts) : "n/a"));
  st.appendChild(row("New entries", s.halt ? "HALTED" : "running", s.halt ? "bad" : "good"));
  var b = el("button", s.halt ? "Resume new entries" : "Halt new entries", s.halt ? "" : "red");
  b.onclick = function(){ api("/api/halt?on=" + (s.halt ? 0 : 1), "POST").then(load).catch(err); };
  st.appendChild(b);

  var op = clear("open"), list = hb.open || [];
  if (!list.length) op.appendChild(el("div", "None", "hint"));
  list.forEach(function(p){
    op.appendChild(row(short(p.mint) + (p.half ? " (half sold)" : ""), pc(p.pct) + " / " + Math.round(p.ageSec / 60) + "m", p.pct >= 0 ? "good" : "bad"));
  });

  var sx = clear("stats"), t = hb.stats || {};
  sx.appendChild(row("Signals gated / passed", (t.gated || 0) + " / " + (t.passed || 0)));
  sx.appendChild(row("Closed trades", String(t.closed || 0)));
  if (t.closed) {
    sx.appendChild(row("Win rate", Number(t.winRate).toFixed(1) + "%"));
    sx.appendChild(row("Mean / median pnl", pc(t.mean) + " / " + pc(t.median), t.median >= 0 ? "good" : "bad"));
    sx.appendChild(row("Best / worst", pc(t.best) + " / " + pc(t.worst)));
    Object.keys(t.byReason || {}).forEach(function(k){
      sx.appendChild(row("  " + k, "n=" + t.byReason[k].n + "  " + pc(t.byReason[k].mean), "hint"));
    });
  }
  if (t.lagMedian !== null && t.lagMedian !== undefined) sx.appendChild(row("Relay lag (median)", Math.round(t.lagMedian / 1000) + "s"));

  var gx = clear("gates"), gs = (hb.gates || []).slice().reverse();
  if (!gs.length) gx.appendChild(el("div", "None yet", "hint"));
  gs.forEach(function(g){
    gx.appendChild(row(short(g.mint), g.pass ? "PASS" : "FAIL", g.pass ? "good" : "bad"));
    if (!g.pass && g.reasons && g.reasons.length) gx.appendChild(el("div", g.reasons.join("; "), "hint small"));
  });

  if (!rulesShown) {
    var rx = clear("rules");
    RULES.forEach(function(r){
      var w = el("div"); w.appendChild(el("label", r[1]));
      var i = el("input"); i.type = "number"; i.id = "r_" + r[0]; i.value = (s.rules || {})[r[0]]; w.appendChild(i); rx.appendChild(w);
    });
    rulesShown = true;
  }
  $("foot").textContent = "Paper mode only. Data is the phone's last heartbeat (every 3 min).";
}
function err(e){ $("foot").textContent = "Error: " + e.message; }
function load(){ return api("/api/state").then(render).catch(err); }
$("save").onclick = function(){
  var body = {};
  RULES.forEach(function(r){ body[r[0]] = Number($("r_" + r[0]).value); });
  api("/api/rules", "POST", body).then(function(){ rulesShown = false; $("foot").textContent = "Saved. Phone applies it within seconds."; return load(); }).catch(err);
};
load(); setInterval(load, 15000);
</script></body></html>`;

export default {
  async fetch(req, env) {
    const url = new URL(req.url);
    const path = url.pathname;
```

```
    // Mini App page (no secrets inside; all data comes from authenticated /api calls).
    if (path === "/app" && req.method === "GET") {
      return new Response(APP_HTML, { headers: { "content-type": "text/html; charset=utf-8", "cache-control": "no-store" } });
    }

    // Mini App API: Telegram initData must validate AND belong to the owner.
    if (path.startsWith("/api/")) {
      if (!(await verifyInit(env, req.headers.get("x-init")))) return json({ error: "unauthorized" }, 401);
      if (path === "/api/state" && req.method === "GET") {
        const [ctl, hb] = await Promise.all([readCtl(env), env.KV.get("hb", "json")]);
        return json({ halt: ctl.halt, rules: { ...(hb?.rules ?? {}), ...ctl.rules }, hb: hb ?? {} });
      }
      if (path === "/api/halt" && req.method === "POST") {
        const ctl = await setHalt(env, url.searchParams.get("on") === "1");
        return json({ halt: ctl.halt });
      }
      if (path === "/api/rules" && req.method === "POST") {
        const rules = cleanRules(await req.json().catch(() => ({})));
        const ctl = await readCtl(env);
        ctl.rules = { ...ctl.rules, ...rules };
        await env.KV.put("ctl", JSON.stringify(ctl));
        return json({ rules: ctl.rules });
      }
      return json({ error: "not found" }, 404);
    }

    // Helius webhook (its own auth header).
    if (path === "/hook" && req.method === "POST") {
      if (req.headers.get("authorization") !== env.HELIUS_AUTH) return json({ error: "unauthorized" }, 401);
      const body = await req.json().catch(() => []);
      const txs = Array.isArray(body) ? body : [body];
      const q = await readQ(env);
      let changed = false;
      for (const tx of txs) {
        for (const mint of extractMints(tx)) {
          if (q.items.some((i) => i.mint === mint)) continue;
          q.seq += 1;
          q.items.push({ id: q.seq, mint, sig: tx.signature ?? "", ts: Date.now() });
          changed = true;
        }
      }
      if (changed) {
        q.items = q.items.slice(-QMAX);
        await env.KV.put("queue", JSON.stringify(q));
      }
      return json({ ok: true });
    }

    // Phone-facing endpoints require the app key.
    if (req.headers.get("x-key") !== env.APP_KEY) return json({ error: "unauthorized" }, 401);

    if (path === "/signals") {
      const since = Number(url.searchParams.get("since") ?? -1);
      const [q, ctl] = await Promise.all([readQ(env), readCtl(env)]);
      const signals = since < 0 ? [] : q.items.filter((i) => i.id > since);
      return json({ signals, seq: q.seq, halt: ctl.halt, rules: ctl.rules });
    }

    if (path === "/heartbeat" && req.method === "POST") {
      const b = await req.json().catch(() => ({}));
      await env.KV.put("hb", JSON.stringify({ ...b, ts: Date.now() }));
      return json({ ok: true });
    }

    if (path === "/notify" && req.method === "POST") {
      const b = await req.json().catch(() => ({}));
      await tg(env, b.text ?? "");
      return json({ ok: true });
    }

    // Kill switch via curl: POST /halt?on=1 stops new entries, on=0 resumes.
    if (path === "/halt" && req.method === "POST") {
      const ctl = await setHalt(env, url.searchParams.get("on") === "1");
      return json({ halt: ctl.halt });
    }

    return json({ error: "not found" }, 404);
  },

  // Dead-man's switch: runs every minute via cron.
  async scheduled(_event, env) {
    const hb = await env.KV.get("hb", "json");
    if (!hb) return;
    const late = Date.now() - hb.ts > HB_LATE_MS;
    const alerted = await env.KV.get("alerted");
    if (late && !alerted) {
      const mins = Math.round((Date.now() - hb.ts) / 60000);
      await tg(env, `Phone silent for ${mins} min. Open paper positions at last heartbeat: ${hb.open?.length ?? 0}`);
      await env.KV.put("alerted", "1", { expirationTtl: 3600 });
    } else if (!late && alerted) {
      await env.KV.delete("alerted");
    }
  },
};
```